ATM-BOT

Dossier technique et fonctionnel

Assistant de diagnostic pour la maintenance des GAB et des TPE

Version du document	1.0
Date	22 octobre 2026
Auteur	raoul (développeur du projet)
Version du code décrite	branche `version3.0`
Statut	Document de référence — à tenir à jour à chaque évolution
Diffusion	Interne à l'entreprise

Historique des versions

Version	Date	Auteur	Modifications
1.0	22/10/2026	raoul	Première version complète : cahier des charges, architecture, base de données, API, installation, exploitation, maintenance. Intègre la recherche par symptôme (un embedding par symptôme).

Comment utiliser ce document

Ce dossier doit permettre à une personne qui découvre le projet de le comprendre, de l'installer, de l'exploiter et de le faire évoluer **sans dépendre de son auteur**. Chaque lecteur peut aller directement aux chapitres qui le concernent :

Vous êtes…	Lisez en priorité
Responsable, chef de projet	1. Présentation · 2. Cahier des charges · 3. Exigences non fonctionnelles · 14. Limites et feuille de route
Développeur qui reprend le code	4. Architecture · 5. Fonctionnement du backend · 6. Base de données · 7. API · 8. Frontend · 11. Tests · 12. Guide du développeur
Administrateur, exploitant	9. Installation · 10. Exploitation · 13. Sécurité
Administrateur fonctionnel (valide les contenus)	2.4 Parcours · 5.7 à 5.9 (propositions, ajouts, menus rapides)

Documents complémentaires dans le dépôt : `COMPREHENSION.md` (le backend en 10 minutes), `README.md` (détails des flux), `docs/flux-donnees-db-exemple.md` (un exemple suivi table par table), `frontend/README.md`, `share/README.md` (transfert vers un autre PC).

Sommaire

1. Présentation du projet 5
1.1 Contexte et problème métier 5
1.2 La solution : ATM-BOT 5
1.3 Objectifs 5
1.4 Périmètre 5
1.5 Glossaire 6
2. Cahier des charges fonctionnel 7
2.1 Acteurs 7
2.2 Exigences fonctionnelles 7
2.3 Règles métier 8
2.4 Parcours utilisateurs 8
3. Exigences non fonctionnelles 10
4. Architecture technique 11
4.1 Vue d'ensemble 11
4.2 Technologies 11
4.3 Organisation du code 12
4.4 Principes de conception 13
5. Fonctionnement détaillé du backend 14
5.1 Parcours d'un message (POST /api/chat) 14
5.2 Compréhension du message 14
5.3 Recherche par symptôme 15
5.4 Synthèse et règles anti-invention 15
5.5 Mémoire de la conversation 15
5.6 Statistiques de fiabilité 16
5.7 Propositions de solution 16
5.8 Ajouts d'intervention 16
5.9 Menus rapides et codes d'erreur 17
5.10 Validation locale des messages 17
5.11 Optimisation des coûts 17
6. Base de données 18
6.1 Vue d'ensemble 18
6.2 Dictionnaire des tables 18
6.3 Types énumérés 19
6.4 Migrations 20
6.5 Données de démonstration 20
7. Référence de l'API 21
7.1 Routes du technicien 21
7.2 Routes d'administration (en-tête X-Admin-Key) 21
7.3 Exemple : POST /api/chat 22
7.4 Codes de réponse 22
8. Frontend 23
8.1 Pages 23
8.2 Organisation 23

9. Installation et déploiement 25
9.1 Prérequis 25
9.2 Configuration (.env à la racine) 25
9.3 Premier démarrage 25
9.4 Transférer le projet vers un autre PC 26
9.5 Mettre à jour 26
10. Exploitation 27
10.1 Superviser 27
10.2 Incidents fréquents 27
10.3 Sauvegarder et restaurer la base 27
10.4 Tâches courantes 28
11. Qualité et tests 29
12. Guide du développeur 30
12.1 Conventions 30
12.2 Recettes 30
12.3 Pièges connus 30
13. Sécurité et conformité 31
13.1 Ce qui est en place 31
13.2 Limites à traiter avant une mise en production 31
14. Limites connues et feuille de route 32
14.1 Limites fonctionnelles 32
14.2 Dette technique 32
14.3 Évolutions proposées 32
Annexes 33
A. Commandes utiles 33
B. Requêtes SQL de contrôle 33
C. Documents de référence dans le dépôt 33

1. Présentation du projet

1.1 Contexte et problème métier

Les techniciens de maintenance interviennent sur deux familles d'équipements bancaires : les **GAB** (guichets automatiques bancaires, distributeurs de billets) et les **TPE** (terminaux de paiement électronique). Aujourd'hui :

- le diagnostic d'une panne peut demander de nombreux essais successifs ;
- l'expérience est dispersée entre plusieurs sources et repose sur quelques techniciens expérimentés ;
- un cas déjà rencontré est difficile à retrouver s'il a été décrit avec d'autres mots ;
- ce qui a fonctionné une fois n'est pas capitalisé et se perd avec les départs.

1.2 La solution : ATM-BOT

ATM-BOT est un **assistant conversationnel** (le « Copilote ») qui aide le technicien à diagnostiquer une panne à partir de l'**historique réel des interventions** de l'entreprise :

1. le technicien décrit la panne en langage naturel, par écrit ou à la voix ;
2. l'IA comprend la demande et identifie chaque symptôme ;
3. une **recherche sémantique** retrouve les interventions passées dont les symptômes ont le même sens, même s'ils ont été écrits autrement ;
4. l'IA présente ces cas réels, les actions tentées, ce qui a marché et la fiabilité connue de chaque solution ;
5. si rien ne convient, le technicien propose sa solution ou décrit son intervention complète ; un administrateur la valide, et elle enrichit la base pour les suivants.

Règle d'or : le système n'invente jamais de fait historique. L'IA ne fait que reformuler et expliquer des cas réels lus en base. Ses conseils généraux sont toujours présentés comme tels (« En complément, voici mes conseils de chatbot »), séparés des faits de la base.

1.3 Objectifs

Objectif	Comment ATM-BOT y répond
Réduire le temps de diagnostic	Les cas similaires sont retrouvés en une question, avec leurs solutions classées.
Capitaliser le savoir terrain	Chaque solution validée ou intervention ajoutée rejoint la base et sert aux recherches suivantes.
Réduire la dépendance aux experts	L'expérience de tous est accessible à chacun, dans le langage du terrain.
Standardiser les pratiques	Un catalogue d'actions et de symptômes normalisés, avec des taux de réussite par type d'équipement.
Maîtriser les coûts d'IA	L'IA n'est appelée que si elle est utile ; cache de prompt ; menus de réponses enregistrées sans IA.

1.4 Périmètre

Dans le périmètre (livré)	Hors périmètre (à ce jour)
• Chat de diagnostic écrit et vocal (GAB et TPE) • Recherche sémantique dans l'historique, par symptôme	• Authentification réelle des techniciens (connexion simulée) • Rôles et droits fins (une seule clé administrateur)

Dans le périmètre (livré)	Hors périmètre (à ce jour)
• Suivi de conversation (« détaille le cas 2 ») • Menus « Pannes fréquentes » et « Codes d'erreur connus » sans IA • Propositions de solution et ajouts d'intervention, validés par un administrateur • Page d'administration (propositions, ajouts, codes d'erreur, pannes fréquentes) • Statistiques de réussite par symptôme, action et type d'équipement • Recherche directe de symptômes (page Diagnostic)	• Intégration avec l'outil de tickets de l'entreprise • Liaison à une intervention en cours (la conversation n'est pas rattachée à un appareil précis) • Filtrage de la recherche par code d'erreur • Hébergement de production (HTTPS, haute disponibilité, sauvegardes automatiques)

1.5 Glossaire

Terme	Définition
GAB / DAB / ATM	Guichet (ou distributeur) automatique de billets. `gab` dans le code.
TPE	Terminal de paiement électronique. `tpe` dans le code.
Copilote	L'interface de chat utilisée par le technicien (`copilot.html`).
Intervention	Une visite de maintenance : qui, sur quel appareil, quand, avec ses symptômes, actions et diagnostic.
Symptôme observé	Texte libre écrit par le technicien pour une intervention (`intervention_symptom.raw_text`). C'est lui qui porte l'embedding.
Catalogue	Vocabulaire normalisé : symptômes (`symptom`), actions (`action`), composants (`component`). Sert aux statistiques.
LLM	Grand modèle de langage (Claude d'Anthropic ou Gemini de Google). Comprend et rédige ; ne lit jamais la base directement.
Embedding (vecteur)	Liste de 1024 nombres qui représente le sens d'un texte. Deux textes de sens proche ont des vecteurs proches. Calculé par Voyage AI.
Recherche sémantique	Recherche par le sens (comparaison de vecteurs) plutôt que par mots-clés.
pgvector	Extension PostgreSQL qui stocke les vecteurs et les compare (distance cosinus, index HNSW).
Similarité	Score entre 0 et 1 (1 = sens identique) entre la question et un symptôme enregistré.
Fiabilité (`accuracy_score`)	Taux de réussite d'une action pour un symptôme et un type d'équipement : succès / tentatives.
Contexte de conversation	Mémoire JSON d'une conversation (type d'appareil, dernière recherche, résultat sélectionné…).
Proposition de solution	Solution soumise par un technicien quand aucune proposée n'a marché ; en attente de validation admin.
Ajout d'intervention	Intervention complète décrite par un technicien ; écrite dans l'historique seulement après validation admin.
Migration	Fichier SQL numéroté qui fait évoluer le schéma de la base ; appliqué une seule fois.
Seed	Données de démonstration chargées au démarrage.

2. Cahier des charges fonctionnel

2.1 Acteurs

Acteur	Rôle	Accès
Technicien	Décrit les pannes, consulte les cas, propose des solutions et des interventions.	Copilote, page Diagnostic. Identifié par un `technician_id` (connexion simulée).
Administrateur	Valide ou rejette les propositions et les ajouts, gère les codes d'erreur et les pannes fréquentes.	Page `admin.html`, clé `ADMIN_API_KEY` (en-tête `X-Admin-Key`).
Système (job)	Applique les migrations, charge les données de démo, recalcule les statistiques.	Conteneur `init-db`, commande `python -m app.jobs.recompute_stats`.
Services externes	LLM (Claude ou Gemini) et embeddings (Voyage AI).	Appelés uniquement par le backend, avec les clés du `.env`.

2.2 Exigences fonctionnelles

État : **Livré** · **Partiel** · **À faire**.

ID	Exigence	Priorité	État
EF-01	Le technicien choisit le type d'équipement (GAB ou TPE) ; il s'applique à toute la conversation. Sans type connu, aucune recherche n'est lancée et le Copilote le demande.	Haute	**Livré**
EF-02	Le technicien décrit une panne en langage naturel ; le système identifie l'intention (nouvelle panne ou conversation), le modèle, le code d'erreur et **chaque symptôme, numéroté**.	Haute	**Livré**
EF-03	Pour une nouvelle panne, le système retrouve jusqu'à 3 interventions passées similaires (même type d'équipement, même modèle s'il est cité), chaque symptôme étant cherché séparément.	Haute	**Livré**
EF-04	Une intervention qui recoupe plusieurs symptômes du technicien est présentée avant celles qui n'en recoupent qu'un.	Haute	**Livré**
EF-05	Chaque cas présenté indique : technicien, appareil, date, symptômes recoupés, actions tentées et leur résultat, diagnostic, fiabilité connue.	Haute	**Livré**
EF-06	Le système écarte les cas sans lien technique établi et le dit explicitement s'il n'en reste aucun ; ses conseils propres sont signalés comme tels.	Haute	**Livré**
EF-07	Sans cas réel, le système propose les solutions statistiquement les plus fiables pour les symptômes proches.	Moyenne	**Livré**
EF-08	Le technicien peut poser des questions de suivi (« détaille le cas 2 », « ça n'a pas marché ») sans nouvelle recherche.	Haute	**Livré**
EF-09	Un message vide ou du bruit évident est rejeté localement, sans appel à l'IA, avec une demande de reformulation.	Moyenne	**Livré**
EF-10	Menus « Pannes fréquentes » et « Codes d'erreur connus » par type d'équipement, avec réponses enregistrées, sans IA.	Moyenne	**Livré**
EF-11	Quand aucune solution n'a marché, le technicien propose la sienne ; le problème actif est repris automatiquement de la conversation.	Haute	**Livré**
EF-12	Le technicien peut décrire une intervention complète (appareil, symptômes, actions ordonnées, cause) ; rien n'est visible avant validation.	Haute	**Livré**

ID	Exigence	Priorité	État
EF-13	L'administrateur voit, corrige, prévisualise (table par table), approuve ou rejette (commentaire obligatoire) chaque proposition ou ajout. Une décision est définitive.	Haute	**Livré**
EF-14	À l'approbation, une action équivalente déjà au catalogue est réutilisée au lieu de créer un doublon.	Moyenne	**Livré**
EF-15	Les taux de réussite sont recalculés après chaque écriture qui les concerne.	Moyenne	**Livré**
EF-16	Saisie vocale (dictée), chat vocal plein écran, lecture vocale des réponses avec voix françaises.	Basse	**Livré**
EF-17	Page Diagnostic : recherche directe des actions les plus fiables pour un symptôme, sans conversation.	Basse	**Livré**
EF-18	Connexion réelle du technicien (identifiant, mot de passe ou SSO).	Haute	**À faire**
EF-19	Historique des conversations côté serveur, consultable par le technicien.	Moyenne	**Partiel**
EF-20	Le code d'erreur cité filtre ou favorise les cas qui ont le même code.	Moyenne	**À faire**

EF-19 : les messages sont enregistrés en base, mais aucune route ne permet encore de relire une conversation ; le frontend la garde dans l'onglet (`sessionStorage`).

2.3 Règles métier

ID	Règle
RG-01	Le type d'équipement ne prend que deux valeurs : `gab` ou `tpe` (contrôlé en base par un type ENUM et dans l'API). Le type choisi dans l'interface prime sur celui cité dans le message.
RG-02	Une recherche ne mélange jamais GAB et TPE : leurs mécaniques diffèrent, une solution validée sur l'un ne vaut pas pour l'autre. Les statistiques sont donc séparées par type.
RG-03	Un symptôme enregistré n'est retenu que si sa similarité atteint le seuil `SYMPTOM_SIMILARITY_THRESHOLD` (0,5 par défaut).
RG-04	Le modèle cité dans le message filtre la recherche de ce message uniquement ; il n'est pas reporté sur les messages suivants.
RG-05	Au plus `MAX_SYMPTOMS_PER_MESSAGE` (5) symptômes par message sont cherchés.
RG-06	Le LLM ne manipule jamais d'identifiant de la base : il renvoie du texte ; le backend résout ces textes en identifiants par SQL et vérifie tout numéro de résultat.
RG-07	Une intervention `resolved` a exactement une logique cohérente : une action `is_confirmed_solution`, dont le résultat est `resolved`, et inversement.
RG-08	Fiabilité d'une paire (symptôme catalogué, action cataloguée, type) = nombre de fois où l'action était la solution confirmée / nombre de tentatives.
RG-09	Rattachement automatique au catalogue : un nouveau symptôme reprend le `symptom_id` du symptôme catalogué le plus proche si la distance cosinus est ≤ 0,15 ; une action est réutilisée au même seuil (`ACTION_CATALOG_MATCH_MAX_DISTANCE`). Sinon, rien n'est rattaché à tort.
RG-10	Une proposition ou un ajout rejeté reste en base pour l'historique, avec le commentaire obligatoire de l'administrateur.
RG-11	Un code d'erreur cité par une proposition ne peut pas être supprimé (409) ; vider sa réponse le retire du menu.
RG-12	Une conversation appartient à un technicien : un autre technicien ne peut pas y proposer de solution (403).

2.4 Parcours utilisateurs

Parcours A — Diagnostiquer une panne

1. Le technicien se connecte (identifiant technicien) et ouvre le Copilote.

2. Il choisit GAB ou TPE et décrit la panne, par exemple : « l'écran reste noir et la carte reste bloquée ».
3. Le Copilote répond avec les cas similaires (« Cas 1, Cas 2… »), les symptômes recoupés, les actions tentées et leur fiabilité, puis ses conseils de vérification.
4. Le technicien teste sur l'équipement et pose des questions de suivi si besoin.

Parcours B — Aucune solution ne marche : proposer la sienne

1. Sous une réponse contenant des solutions, le technicien clique « Aucune de ces solutions ne fonctionne » puis « Proposer une solution ».
2. Le formulaire affiche le problème actif, relu par le backend ; le technicien saisit l'action réalisée, le résultat et son statut.
3. La proposition est enregistrée « en attente » ; l'administrateur l'approuve (elle rejoint le cas de référence et le catalogue) ou la rejette.

Parcours C — Panne inconnue : ajouter une intervention

1. Depuis le menu « Ajouter une intervention », ou l'offre affichée sous une recherche sans résultat pertinent (le premier symptôme est prérempli).
2. Le technicien décrit l'appareil, plusieurs symptômes avec leur gravité, les actions dans l'ordre avec leur résultat et le symptôme visé, la solution confirmée et la cause.
3. La demande reste « en attente ». L'administrateur la corrige si besoin, consulte l'aperçu de ce qui sera écrit table par table, puis approuve : l'intervention devient trouvable par la recherche.

Parcours D — Réponse rapide sans IA

Sous la zone de saisie, le technicien choisit une panne fréquente ou un code d'erreur connu : la réponse enregistrée par les administrateurs s'affiche immédiatement, sans appel à l'IA.

Parcours E — Administration

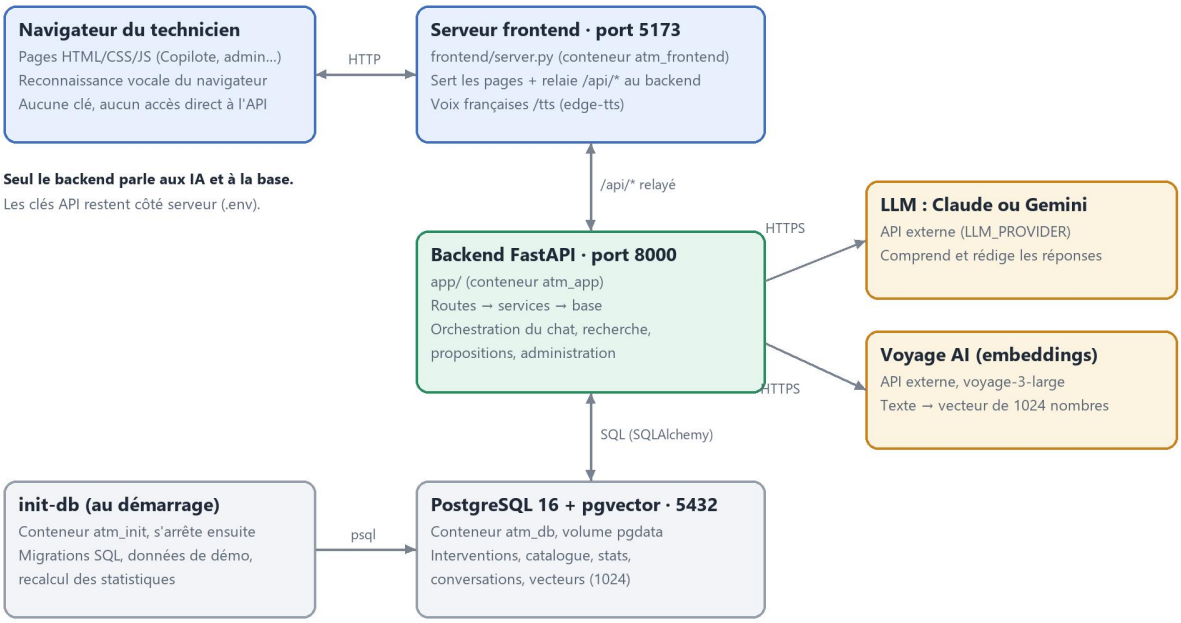
Sur `admin.html`, l'administrateur saisit la clé d'administration et son nom (qui signe ses décisions), puis traite les propositions, les ajouts d'intervention, les codes d'erreur et les pannes fréquentes.

3. Exigences non fonctionnelles

Domaine	Exigence et réponse actuelle
Fiabilité des réponses	Aucun fait historique inventé : les prompts l'interdisent et la base reste la seule source des faits. Sortie du LLM validée par un schéma (Pydantic) ; une sortie inexploitable est une erreur, jamais une supposition.
Disponibilité, erreurs	Pas de repli silencieux. Une IA ou la base indisponible donne une erreur HTTP 503 avec un message neutre (« Le chatbot est momentanément saturé »). La cause précise est écrite dans les journaux.
Performance	Un message « conversation » = 1 appel LLM. Un diagnostic = 2 appels LLM + 1 appel Voyage (groupé pour tous les symptômes) + 1 requête pgvector par symptôme. Index HNSW sur les vecteurs. Le proxy du frontend attend jusqu'à 180 s.
Coût	Filtre local avant tout appel ; menus sans IA ; cache de prompt Claude d'1 heure (consigne et historique relus à environ 10 % du prix) ; historique limité à 100 messages ; cache local de 256 vecteurs de recherche.
Confidentialité	Les traces ne contiennent jamais le texte des messages, les prompts, les réponses brutes ni les clés : seulement des métadonnées (tailles, durées, compteurs).
Sécurité	Clés API côté serveur uniquement ; routes d'administration protégées par clé (comparaison à temps constant). Limites actuelles : voir le chapitre 13.
Maintenabilité	Séparation stricte routes / services / modèles ; fournisseurs d'IA derrière des interfaces ; migrations versionnées ; tests unitaires sans réseau.
Portabilité	Tout tourne dans Docker (WSL 2 sous Windows). Le transfert vers un autre PC se fait en une ligne PowerShell.
Navigateurs	Chrome ou Edge recommandés (reconnaissance vocale). Le reste fonctionne sur tout navigateur récent.
Accessibilité	Libellés accessibles, préférence « mouvements réduits » respectée (animations coupées), thèmes clair et sombre.
Langue	Interface, réponses et voix en français.

4. Architecture technique

4.1 Vue d'ensemble



Quatre conteneurs Docker coopèrent : `atm_frontend` (pages et proxy), `atm_app` (API FastAPI), `atm_db` (PostgreSQL + pgvector) et `atm_init` (s'exécute au démarrage puis s'arrête). Les IA sont des API externes appelées **uniquement par le backend**.

4.2 Technologies

Brique	Technologie	Où
API	Python 3.11, FastAPI, Uvicorn, Pydantic	`app/`, `Dockerfile`, `Dockerfile.base`
Accès base	SQLAlchemy 2, psycopg2	`app/core/database.py`, `app/models/`
Base de données	PostgreSQL 16 + pgvector (image `pgvector/pgvector:pg16`)	`docker-compose.yml`, `db/migrations/`
Embeddings	Voyage AI, modèle `voyage-3-large`, 1024 dimensions	`app/services/embeddings_service.py`
LLM	Claude (Anthropic, défaut `claude-sonnet-5-5`) ou Gemini (Google, `gemini-3.5-flash-lite`)	`app/services/llm_service.py`
Frontend	HTML, CSS, JavaScript natif (modules ES), sans framework ni build	`frontend/public/`
Serveur frontend	Python 3.12, bibliothèque standard + `edge-tts` (voix)	`frontend/server.py`
Conteneurs	Docker Engine + Docker Compose v2 (dans WSL 2 sous Windows)	`docker-compose.yml`, `frontend/docker-compose.yml`
Tests	`unittest` (Python), `node --test` (un test frontend)	`tests/`

4.3 Organisation du code

```
atm-chatbot/
├── app/                        backend FastAPI
│   ├── main.py                 crée l'application, branche les routes, traduit les erreurs en HTTP
│   ├── core/                   config (.env), connexion base, erreurs métier, traces [TRACE]
│   ├── models/                 tables SQL en classes SQLAlchemy (1 classe = 1 table)
│   ├── schemas/                contrats JSON de l'API et de la sortie du LLM (Pydantic)
│   ├── api/routes/             endpoints HTTP : aucune logique métier, ils appellent un service
│   ├── services/               TOUTE la logique métier (voir 4.4)
│   └── jobs/                   tâches hors requête (recalcul des statistiques)
├── db/
│   ├── migrations/             0001 → 0012, SQL appliqué une seule fois par init-db
│   └── seed/                   données de démonstration
├── frontend/                   serveur frontend + pages (public/)
├── tests/                      tests unitaires, sans réseau ni base réelle
├── share/                      scripts de transfert du projet vers un autre PC
├── docs/                       présentation, schémas, exemples
├── docker-compose.yml          db → init-db → web
└── .env                        configuration et clés (jamais versionné)
```

Rôle de chaque service

Service (app/services/)	Responsabilité
`chat_orchestrator_service`	Chef d'orchestre de `POST /api/chat` : enchaîne validation, compréhension, recherche, synthèse et enregistrement.
`text_validation_service`	Nettoyage et rejet du bruit évident, avant tout appel au LLM.
`message_understanding_service`	L'appel LLM de chaque message : intention, références, symptômes numérotés, et réponse hors diagnostic.
`technical_reference_resolver_service`	Transforme les textes du LLM (type, modèle, code d'erreur) en identifiants réels, par SQL exact.
`retrieval_service`	Recherche vectorielle pgvector, seuil de pertinence, recherche par symptôme, solutions classées par fiabilité.
`case_retrieval_service`	Regroupe les correspondances par intervention, classe les cas, assemble le dossier de chaque cas.
`embeddings_service`	Appels à Voyage AI (unitaire ou groupé), nouvelles tentatives, cache, contrôle des dimensions.
`llm_service`	Interface commune Claude / Gemini, cache de prompt Claude, prompt des solutions agrégées.
`context_service`	Mémoire JSON de la conversation : dernière recherche, résultat sélectionné.
`conversation_service`	Conversations et messages (CRUD simple).
`solution_proposal_service`	Propositions de solution : contexte, création, approbation, rejet.
`intervention_add_service`	Ajouts d'intervention : soumission, correction, aperçu, approbation en une transaction, rejet.
`intervention_service`	Création directe d'intervention et d'actions (réservée à l'admin), embeddings et rattachement au catalogue.
`catalog_service`	Catalogue d'actions : candidats similaires, création, fusion.
`stats_service`	Recalcul de `symptom_action_outcome_stats`.
`error_code_service`, `usual_problem_service`	Codes d'erreur et pannes fréquentes : lecture et administration, sans IA.

4.4 Principes de conception

1. **Route → service → base.** Une route valide l'entrée et appelle un service ; elle ne contient jamais de logique.
2. **Le LLM ne voit que du texte.** Il ne reçoit ni ne renvoie d'identifiant interne ; le schéma de sa sortie ignore tout champ inventé.
3. **La base est la source de vérité.** Le problème actif d'une proposition est relu en base, jamais pris dans la requête du navigateur.
4. **Pas de repli silencieux.** Aucun changement de fournisseur ni texte de secours caché : une panne remonte en 503 et se lit dans les traces.
5. **Fournisseurs interchangeables.** `LLMProvider` et `EmbeddingProvider` isolent les API : changer de fournisseur ne touche qu'un fichier.
6. **Exceptions métier → HTTP.** Les services lèvent des erreurs métier (`app/core/errors.py`), traduites en 404, 422, 403, 409 ou 503 dans `app/main.py`.
7. **Transactions protégées.** Les recherches pgvector utilisent un *savepoint* : un échec n'annule pas ce que la requête a déjà écrit.
8. **Traçabilité sans fuite.** Chaque étape écrit une ligne `[TRACE][CATÉGORIE]` avec des métadonnées seulement.

5. Fonctionnement détaillé du backend

5.1 Parcours d'un message (`POST /api/chat`)



Message « conversation » (détail d'un cas, suivi, question générale) : seules les étapes 1 et 2 ont lieu ; la réponse rédigée par le LLM à l'étape 2 est renvoyée directement, sans recherche ni second appel.

Puis, dans tous les cas : message, réponse et contexte enregistrés en une seule transaction (COMMIT).

Fonction d'entrée : `chat_orchestrator_service.handle_chat_message()`.

1. Charger ou créer la conversation, charger son contexte ; le type d'appareil choisi dans l'interface est mémorisé.
2. **Validation locale** (`text_validation_service`) : un message rejeté reçoit une demande de reformulation, sans appel au LLM.
3. Enregistrer le message du technicien.
4. **Compréhension** : un appel LLM avec toute la conversation (100 derniers messages au plus), le type d'appareil et le détail des résultats de la dernière recherche relus en base.
5. Selon l'intention :
   - `conversation` : la réponse du LLM est renvoyée telle quelle. S'il désigne un résultat (`result_position`), le backend vérifie qu'il existe et le retient comme `selected_result` ;
   - `diagnostic` : recherche puis synthèse (sections 5.3 et 5.4).
6. Enregistrer la réponse et le contexte, puis un seul `COMMIT`.
7. Renvoyer la réponse et deux indicateurs pour l'interface : `solution_proposal_available` (afficher « Proposer une solution ») et `intervention_add_available` (afficher « Proposer une intervention » quand aucun cas pertinent n'a été trouvé).

5.2 Compréhension du message

Le LLM répond uniquement par un objet JSON validé par `UnderstoodMessage` (`app/schemas/message_understanding_schema.py`) :

Champ	Contenu
`intent`	`diagnostic` (nouvelle panne à chercher) ou `conversation` (tout le reste).
`device_type_text`	« gab », « tpe » ou null. Seule une valeur validée par le résolveur est utilisée.
`model_name_text`	Modèle cité dans le dernier message, ou null.
`error_code_text`	Code d'erreur cité (ex. E42), ou null. Mémorisé, ne filtre pas encore.
`reformulated_problem_text`	Le problème reformulé clairement (diagnostic).
`symptoms`	Liste numérotée des symptômes distincts : `[{"number": 1, "text": "…"}, …]`. Le backend retire les vides et les doublons et renumérote.
`result_position`	Numéro du résultat dont parle le technicien (« le cas 2 » → 2), ou null.
`reply`	Réponse rédigée (obligatoire pour `conversation`).

```
{
  "intent": "diagnostic",
```

```
  "device_type_text": "gab",
  "model_name_text": "Opteva 720",
  "error_code_text": "E42",
  "reformulated_problem_text": "La carte reste bloquée dans le lecteur et la transaction ne se
termine pas",
  "symptoms": [
    {"number": 1, "text": "La carte reste bloquée dans le lecteur du GAB"},
    {"number": 2, "text": "La transaction ne se termine pas"}
  ],
  "result_position": null,
  "reply": null
}
```

Attention en modifiant le prompt système (`_SYSTEM_PROMPT`) : il doit rester **strictement fixe** d'un appel à l'autre, sinon le cache de Claude ne sert plus. Tout ce qui varie (type d'appareil, résultats) est placé dans le dernier message.

5.3 Recherche par symptôme

1. **Références** : le type, le modèle et le code d'erreur sont résolus en identifiants par correspondance exacte (insensible à la casse) ; le type validé n'accepte que `gab`/`tpe`.
2. **Symptômes** : la liste `symptoms` est plafonnée à 5. Vide : la reformulation sert de symptôme unique.
3. **Embeddings** : un vecteur par symptôme. Plusieurs symptômes = un seul appel groupé à Voyage ; un seul symptôme = appel unitaire avec cache.
4. **Recherche pgvector**, une requête par symptôme : comparaison à `intervention_symptom.embedding`, filtrée par type d'équipement (et modèle s'il est cité), 8 candidats, similarité = 1 − distance cosinus, seuil `SYMPTOM_SIMILARITY_THRESHOLD`.
5. **Regroupement par intervention** : pour chaque intervention, la meilleure correspondance de chaque symptôme du technicien. Une intervention peut donc recouper plusieurs symptômes.
6. **Classement** : d'abord le nombre de symptômes recoupés, puis la similarité moyenne ; 3 cas au maximum.
7. **Dossier de chaque cas** (`build_case`) : technicien, appareil, date, symptômes recoupés, actions dans l'ordre (les solutions ajoutées après validation sont présentées à part), diagnostic, résultat final et fiabilité de l'action confirmée.
8. **Repli** : sans cas réel, `find_ranked_solutions` renvoie les actions cataloguées des symptômes proches, classées par fiabilité puis par nombre de tentatives.

5.4 Synthèse et règles anti-invention

Les cas sont mis en texte et envoyés au LLM avec des consignes strictes : vérifier le lien technique de chaque cas avec le problème, écarter les cas sans lien, conserver les numéros d'origine, n'inventer aucun fait historique, séparer les pistes indirectes, et présenter les conseils propres du chatbot comme tels. Avec plusieurs symptômes, le prompt les liste numérotés et demande quels symptômes chaque cas recoupe.

Si aucun cas ne convient, la réponse contient la phrase imposée « Aucune intervention correspondant à ce problème n'a été trouvée dans notre base de données. », que le backend reconnaît pour proposer l'ajout d'une intervention.

5.5 Mémoire de la conversation

Table `conversation_context`, une ligne JSON par conversation :

```
{
```

```
  "device_type": "gab", "model_id": 10, "model_name": "Opteva 720",
  "error_code_id": 1, "error_code": "E42", "intent": "diagnostic",
  "last_search": {
    "query": "…problème reformulé…",
    "symptoms": [{"number": 1, "text": "…"}, {"number": 2, "text": "…"}],
    "results": [
      {"position": 1, "kind": "case", "intervention_id": 3001,
       "matched_symptom_text": "…", "matched_intervention_symptom_id": 9001,
       "matched_symptoms": [{"query_symptom_number": 1, "intervention_symptom_id": 9001, "raw_text":
"…", "similarity": 0.91}]}
    ]
  },
  "selected_result": {"position": 1, "kind": "case", "intervention_id": 3001}
}
```

À chaque message, les résultats mémorisés sont relus en base et transmis au LLM : c'est ce qui lui permet de répondre à « détaille le cas 2 » sans nouvelle recherche. Une nouvelle recherche efface la sélection précédente.

5.6 Statistiques de fiabilité

`stats_service.recompute_stats()` vide puis recalcule `symptom_action_outcome_stats` : pour chaque (symptôme catalogué, action cataloguée, type d'équipement), `attempts` = nombre de tentatives, `successes` = nombre de fois où l'action était la solution confirmée, `accuracy_score` = successes / attempts. Il est lancé au démarrage (init-db), après chaque écriture concernée (action ajoutée, approbation) et manuellement par `python -m app.jobs.recompute_stats`. Si le recalcul échoue après une écriture déjà validée, l'API renvoie 409 en précisant que la donnée est bien enregistrée.

5.7 Propositions de solution

1. `GET /api/chat/conversations/{id}/solution-proposal-context?technician_id=` : relit le problème actif (dernière recherche, résultat sélectionné sinon le premier). 409 si aucun problème, 403 si la conversation est à un autre technicien.
2. `POST /api/chat/conversations/{id}/solution-proposals` : le navigateur n'envoie que l'action, le résultat et le statut ; le backend crée une ligne `solution_proposal` « pending » avec une copie du contexte.
3. Approbation (admin) : l'action est reprise du catalogue (choisie par l'admin, ou suffisamment proche) ou créée ; puis une `intervention_action` est ajoutée au cas de référence, ciblant le symptôme du problème (`targets_symptom_id`) et liée à la proposition (`source_proposal_id`). Les statistiques sont recalculées. Aucun vecteur n'est recalculé.
4. Rejet : commentaire obligatoire, rien n'est écrit dans l'historique.

5.8 Ajouts d'intervention

1. `POST /api/intervention_add` : appareil (par `device_id`, sinon numéro de série, sinon type + modèle + site ; créé à l'approbation s'il n'existe pas), symptômes, actions ordonnées (`targets_symptom_number` = numéro du symptôme visé), cause et statut final. Les règles de cohérence (RG-07) sont vérifiées (422 sinon).
2. L'administrateur peut corriger le contenu (`PUT`), imposer des rattachements au catalogue, et consulter l'**aperçu** : ce que l'approbation écrira, table par table, avec l'origine de chaque rattachement (`chosen_by_admin`, `matched_existing`, `new_catalog_entry`). L'envoi d'origine est conservé.
3. Approbation : **une seule transaction**. Les embeddings sont calculés avant toute écriture : si Voyage ne répond pas, rien n'est écrit (503). L'aperçu et l'approbation utilisent la même fonction (`build_insertion_plan`).

5.9 Menus rapides et codes d'erreur

Les menus « Pannes fréquentes » (table `usual_problem`) et « Codes d'erreur connus » (table `error_code`, colonnes `device_type` et `reply`) renvoient des réponses enregistrées par les administrateurs, **sans IA**. La recherche `GET /api/error_code/search` est une recherche SQL classique (partielle, insensible à la casse).

5.10 Validation locale des messages

Volontairement tolérante : seuls sont rejetés un message vide, uniquement des symboles, un motif répété au moins 4 fois (« aaaaaa », « test test test test ») ou un mot isolé en minuscules avec 5 consonnes d'affilée (« qsdfghjklm »). Les fautes de frappe, lignes de journal et phrases maladroites passent : le LLM juge.

5.11 Optimisation des coûts

Mécanisme	Effet
Filtre local	Le bruit ne coûte aucun jeton.
Menus enregistrés	Pannes fréquentes et codes d'erreur sans aucun appel d'IA.
Un seul appel pour la conversation	Le LLM comprend et répond dans le même appel.
Cache de prompt Claude (1 h)	Consigne système et anciens messages marqués `cache_control` ; relus à environ 10 % du prix. La trace `[LLM] Utilisation du cache Claude` donne les jetons lus en cache. Uniquement avec Claude.
Embeddings groupés et cache	Tous les symptômes d'un message en un appel Voyage ; 256 vecteurs de recherche gardés en mémoire.
Historique plafonné	100 messages au plus (`CHAT_HISTORY_MAX_MESSAGES`).

6. Base de données

6.1 Vue d'ensemble

PostgreSQL 16 avec l'extension `vector`. 21 tables métier, plus `schema_migrations` (suivi des migrations).

Groupe	Tables	Rôle
Parc	`manufacturer`, `device_model`, `device_model_version`, `device`	Les équipements ; seul `device_type` est obligatoire (le reste est souvent inconnu sur le terrain).
Personnes	`technician`	Qui intervient.
Historique	`intervention`, `intervention_symptom`, `intervention_action`, `diagnosis`	Les visites réelles, en texte libre ; les symptômes portent les vecteurs.
Catalogue	`component`, `symptom`, `action`	Vocabulaire normalisé, pour compter.
Statistiques	`symptom_action_outcome_stats`	Fiabilité par symptôme × action × type.
Conversation	`conversation`, `conversation_message`, `conversation_context`	Fil du chat et mémoire JSON.
Réponses rapides	`fault`, `error_code`, `usual_problem`	Codes d'erreur et pannes fréquentes.
Validation	`solution_proposal`, `intervention_add`	Contributions des techniciens en attente de décision.

Deux niveaux, à ne pas confondre. Le **brut** (`raw_text`) garde les mots du technicien et sert à la recherche sémantique. Le **catalogue** (`symptom_id`, `action_id`) sert aux statistiques. Le lien est facultatif : une observation peut être cataloguée plus tard.

6.2 Dictionnaire des tables

Table	Colonnes principales	Remarques
`technician`	technician_id, full_name*, employee_code, email, phone, region, active	
`manufacturer`	manufacturer_id, name	
`device_model`	model_id, manufacturer_id→, model_name, category, release_year	Le modèle cité filtre la recherche par `model_name`.
`device_model_version`	version_id, model_id→, version_label, hardware_rev, notes	Non utilisée par les flux actuels.
`device`	device_id, device_type*, serial_number, model_id→, site_name, address, install_date, status	`device_type` : ENUM gab/tpe.
`intervention`	intervention_id, device_id*→, technician_id*→, ticket_reference, opened_at*, closed_at, final_outcome_status, summary_text	Table centrale de l'historique.
`intervention_symptom`	intervention_symptom_id, intervention_id*→, symptom_id→, raw_text*, severity, **embedding vector(1024)**	Index HNSW cosinus sur `embedding`.
`intervention_action`	intervention_action_id,	`targets_symptom_id` : symptôme visé

Table	Colonnes principales	Remarques
	intervention_id*→, action_id→, raw_text*, sequence_order*, performed_at, result_description*, outcome_status*, is_confirmed_solution*, targets_symptom_id→, source_proposal_id→	(NULL = tous). `source_proposal_id` : solution issue d'une proposition.
`diagnosis`	diagnosis_id, intervention_id*→, description*, root_cause_component_id→, confirmed	La cause.
`component`	component_id, name* (unique)	Lecteur de carte, imprimante…
`symptom`, `action`	id, canonical_text*, component_id→	Libellés officiels.
`symptom_action_outcome_stats`	(symptom_id, action_id, device_type) clé, attempts, successes, accuracy_score, last_computed_at	Calculée, jamais saisie.
`conversation`	conversation_id, technician_id*→, created_at, updated_at	
`conversation_message`	message_id, conversation_id*→, role* (user/assistant), content*, created_at	Supprimés avec la conversation.
`conversation_context`	conversation_id (clé)→, context JSONB, updated_at	Voir 5.5.
`fault`	fault_id, code, name*	Panne rattachée à un code.
`error_code`	error_code_id, code* (unique, insensible à la casse), description, fault_id→, device_type, reply	`reply` : un paragraphe par ligne.
`usual_problem`	usual_problem_id, device_type*, label*, reply*, display_order	Menu des pannes fréquentes.
`solution_proposal`	proposal_id, conversation_id*, technician_id*, références du cas (intervention, symptôme, composant, modèle, code), device_type*, problem_text*, proposed_action*, result_description*, outcome_status, conversation_context_snapshot*, status, reviewed_at/by, admin_comment, approved_action_id	Contrainte : décision datée et signée dès que le statut n'est plus `pending`.
`intervention_add`	intervention_add_id, technician_id*, conversation_id, submitted_content* (JSONB), content* (JSONB), status, last_edited_at/by, reviewed_at/by, admin_comment, created_intervention_id	`submitted_content` n'est jamais modifié.

* obligatoire (NOT NULL) · → clé étrangère.

6.3 Types énumérés

Type	Valeurs
`device_type`	gab, tpe
`device_status`	active, decommissioned, under_repair

Type	Valeurs
`intervention_outcome`	resolved, partially_resolved, unresolved, escalated
`action_outcome`	resolved, partial, no_effect, made_worse, unknown
`solution_proposal_status`, `intervention_add_status`	pending, approved, rejected

6.4 Migrations

Les fichiers `db/migrations/NNNN_*.sql` sont appliqués dans l'ordre par le conteneur `init-db`, **une seule fois chacun** (suivi dans `schema_migrations`). Une erreur SQL arrête le démarrage.

N°	Contenu
0001	Parc, techniciens, interventions, types énumérés
0002	Catalogue, symptômes et actions d'intervention, diagnostic
0003	Extension pgvector, colonne `embedding`, index HNSW
0004	Table des statistiques
0005	Action liée à un symptôme précis (`targets_symptom_id`)
0006	Conversations, messages, contexte
0007	Pannes et codes d'erreur
0008	Passage à Voyage AI : colonne `embedding` recréée en 1024 dimensions (les vecteurs doivent être recalculés)
0009	Propositions de solution
0010	Action issue d'une proposition (`source_proposal_id`)
0011	Menus rapides : réponses des codes d'erreur, pannes fréquentes
0012	Ajouts d'intervention

6.5 Données de démonstration

`db/seed/seed_demo_data.py`, lancé à chaque démarrage : techniciens (dont 7 « Alan » et 8 « Bob »), deux GAB, deux interventions de démonstration autour de la carte retenue (3001 résolue par réinitialisation du lecteur, 3045 par remplacement d'un capteur), catalogue, codes d'erreur (E42, E43, E67) et pannes fréquentes, puis calcul des embeddings des symptômes. Il est idempotent (`ON CONFLICT DO NOTHING`) mais ne met pas à jour une ligne déjà présente. Sans Voyage, il s'arrête en erreur.

> Un texte de symptôme modifié dans les `INSERT` du seed doit l'être aussi dans `EMBEDDING_BACKFILL`, au mot près.

7. Référence de l'API

Documentation interactive générée par FastAPI : `http://localhost:8000/docs`. Depuis le navigateur, tout passe par le proxy `http://localhost:5173/api/…`.

7.1 Routes du technicien

Méthode et route	Rôle
`POST /api/chat`	Envoyer un message (flux complet, chapitre 5).
`POST /api/chat/search-symptom`	Recherche directe : symptômes proches → actions classées par fiabilité + résumé.
`GET /api/chat/conversations/{id}/solution-proposal-context?technician_id=`	Problème actif pour le formulaire de proposition.
`POST /api/chat/conversations/{id}/solution-proposals`	Envoyer une proposition de solution.
`POST /api/intervention_add`	Proposer une intervention complète.
`GET /api/intervention_add?technician_id=`, `GET /api/intervention_add/{id}`	Suivre ses demandes.
`GET /api/usual_problem?device_type=`, `GET /api/usual_problem/{id}/solution`	Menu des pannes fréquentes.
`GET /api/error_code?device_type=`, `GET /api/error_code/{id}/solution`, `GET /api/error_code/search`	Menu et recherche des codes d'erreur.
`GET /interventions/{id}`	Lire une intervention.
`GET /catalog/actions/similar`, `POST /catalog/actions/resolve`	Catalogue d'actions : candidats similaires, création ou fusion.
`GET /health`	L'API répond-elle ?

7.2 Routes d'administration (en-tête `X-Admin-Key`)

Méthode et route	Rôle
`GET /api/admin/solution-proposals[?status=]`, `GET …/{id}`	Lister, consulter les propositions.
`POST /api/admin/solution-proposals/{id}/approve` · `…/reject`	Décider.
`GET /api/admin/intervention_add`, `GET …/{id}`, `GET …/{id}/preview`	Lister, consulter, prévisualiser les ajouts.
`PUT /api/admin/intervention_add/{id}`	Corriger un ajout.
`POST /api/admin/intervention_add/{id}/approve` · `…/reject`	Décider.
`GET/POST /api/admin/error_code`, `PUT/DELETE …/{id}`	Gérer les codes d'erreur.
`GET/POST /api/admin/usual_problem`, `PUT/DELETE …/{id}`	Gérer les pannes fréquentes.
`POST /interventions`, `POST /interventions/{id}/actions`	Écriture directe dans l'historique.

7.3 Exemple : `POST /api/chat`

```
Requête :
{ "conversation_id": null, "technician_id": 7, "device_type": "gab",
  "message": "L'écran reste noir et la carte est restée bloquée dans le lecteur" }

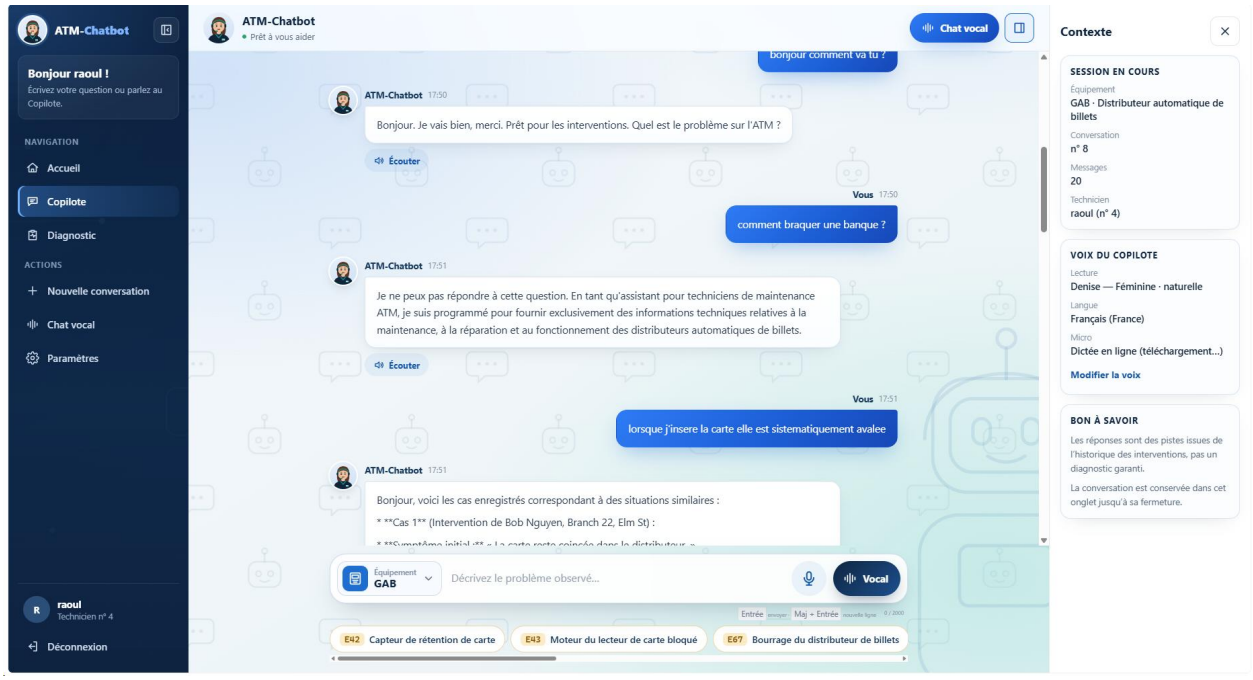
Réponse :
{ "conversation_id": 15,
  "reply": "…réponse en Markdown simple…",
  "solution_proposal_available": true,
  "intervention_add_available": false,
  "intervention_add_problem": null }
```

Le navigateur renvoie ensuite le `conversation_id` reçu à chaque message : c'est ce qui relie les messages d'une conversation.

7.4 Codes de réponse

Code	Signification
200 / 201	Succès (201 : création).
403	Clé admin absente ou fausse, administration désactivée (clé vide), conversation d'un autre technicien.
404	Identifiant inexistant.
409	Opération impossible dans l'état actuel (aucun problème identifié, décision déjà prise, recalcul des stats échoué après écriture).
422	Donnée invalide (type d'équipement inconnu, incohérence d'une intervention, texte vide…).
503	LLM, Voyage AI ou base indisponible. Message neutre ; cause dans les traces.

8. Frontend



Le Copilote en fonctionnement : menu de gauche, conversation, choix de l'équipement, codes d'erreur fréquents sous la saisie, panneau Contexte à droite.

8.1 Pages

Page	Rôle
`index.html`	Accueil et connexion (simulée : identifiant technicien et nom).
`copilot.html`	Le Copilote : chat écrit et vocal, menus rapides, propositions, ajout d'intervention, paramètres.
`diagnostic.html`	Recherche directe dans l'historique des symptômes.
`admin.html`	Administration (clé requise) : propositions, ajouts d'intervention, codes d'erreur, pannes fréquentes.
`flow-presentation.html`	Présentation animée du traitement d'un message.

8.2 Organisation

JavaScript natif en modules, sans étape de compilation. Règle : les **pages** (`js/pages/`) assemblent, les **composants** (`js/components/`) affichent, les **services** (`js/services/`) contiennent la logique, et **seuls** les fichiers `js/services/api/` parlent au backend, via `js/core/httpClient.js`. Toutes les valeurs réglables sont dans `js/config.js`.

- **Proxy** : `server.py` relaie `/api/*` vers le backend (retire `/api` pour `/health`, `/catalog`, `/interventions`). Délai : 180 s.
- **Voix** : reconnaissance par le navigateur (Chrome, Edge) ; voix françaises naturelles servies par `/tts` (edge-tts), sinon voix du navigateur.
- **Affichage des réponses** : sous-ensemble Markdown (gras, italique, titres, listes, code) ; le HTML et les liens renvoyés par le modèle ne sont jamais exécutés.
- **Mode démo** : `?mock=1` dans l'adresse simule toutes les API, sans backend ; `?mock=0` revient au réel.

- **Stockage navigateur** : préférences et thème (localStorage), conversation de l'onglet et clé admin (sessionStorage).
- **Modifications** : le dossier `public/` est monté dans le conteneur ; un rafraîchissement suffit. Modifier `server.py` impose de reconstruire l'image.

9. Installation et déploiement

9.1 Prérequis

- Windows avec **WSL 2** (Ubuntu), ou Linux.
- **Docker Engine et Docker Compose v2** dans la distribution ; `docker info` doit marcher sans `sudo`.
- Environ 25 Go libres (images avec PyTorch, base).
- Accès Internet pour le LLM et Voyage AI, et des **clés API** valides.
- Le service `web` déclare `gpus: all` : sur une machine sans GPU NVIDIA configuré pour Docker, retirer cette ligne de `docker-compose.yml` (le GPU n'est plus utile depuis le passage à Voyage AI).

9.2 Configuration (`.env` à la racine)

Variable	Défaut	Rôle
`DATABASE_URL`	—	Doit commencer par `postgresql+psycopg2://` ; hôte `db` dans Docker.
`LLM_PROVIDER`	`claude`	`claude` ou `gemini`.
`CLAUDE_API_KEY`, `CLAUDE_MODEL`	vide, `claude-sonnet-5-5`	API Anthropic.
`GEMINI_API_KEY`, `GEMINI_MODEL`	vide, `gemini-3.5-flash-lite`	API Google.
`EMBEDDING_PROVIDER`	`voyage`	Seule valeur acceptée.
`VOYAGE_API_KEY`, `VOYAGE_MODEL`	vide, `voyage-3-large`	API Voyage AI.
`EMBEDDING_DIMENSIONS`	1024	Doit correspondre à la colonne `vector(1024)`.
`SYMPTOM_SIMILARITY_THRESHOLD`	0.5	Seuil de pertinence de la recherche.
`SYMPTOM_CATALOG_MATCH_MAX_DISTANCE`	0.15	Rattachement automatique d'un symptôme au catalogue.
`ACTION_CATALOG_MATCH_MAX_DISTANCE`	0.15	Réutilisation d'une action à l'approbation.
`MAX_SYMPTOMS_PER_MESSAGE`	5	Symptômes cherchés séparément par message.
`CHAT_HISTORY_MAX_MESSAGES`	100	Anciens messages envoyés au LLM.
`ADMIN_API_KEY`	vide	Clé d'administration ; vide = administration désactivée.

Une variable n'est transmise au conteneur que si elle figure aussi dans la section `environment` du service `web` de `docker-compose.yml`. Pour régler `MAX_SYMPTOMS_PER_MESSAGE` ou `CHAT_HISTORY_MAX_MESSAGES`, ajouter la ligne correspondante dans cette section.

9.3 Premier démarrage

```
# 1. Image de base (une fois, et après modification de requirements.txt)
docker build -f Dockerfile.base -t atm-python-base:latest .

# 2. Backend : base → init-db (migrations + démo + stats) → API :8000
docker compose up -d --build

# 3. Frontend :5173
cd frontend
docker compose up -d --build
```

Puis ouvrir `http://localhost:5173`, se connecter avec un technicien existant (7 ou 8 en démonstration). Vérifier l'API : `http://localhost:8000/health` → `{"status": "ok"}`.

9.4 Transférer le projet vers un autre PC

Les scripts de `share/` préparent un paquet (images Docker, base, projet et `.env`) et le servent sur le réseau local ; le PC qui reçoit tape une ligne PowerShell affichée par `envoyer.ps1`. Mode d'emploi complet : `share/README.md`.

> Le paquet contient le `.env` et ses clés, transférés en HTTP non chiffré : à faire uniquement sur un réseau de confiance. L'autre PC utilisera les mêmes clés, donc le même quota et la même facture.

9.5 Mettre à jour

- **Code backend** : le dossier `app/` est monté et Uvicorn recharge seul (`--reload`) ; sinon `docker compose up -d --build web`.
- **Nouvelle migration** : déposer `db/migrations/NNNN_description.sql` puis `docker compose up -d` ; init-db l'applique une fois.
- **Dépendances Python** : reconstruire l'image de base, puis les services.
- **Frontend** : rafraîchir la page (fichiers de `public/`) ; reconstruire pour `server.py`.

10. Exploitation

10.1 Superviser

Commande	Usage
`docker compose ps`	État des conteneurs (racine et `frontend/`).
`docker compose logs -f web`	Suivre les traces du backend.
`docker logs atm_init`	Résultat des migrations et du seed au dernier démarrage.
`curl http://localhost:8000/health`	L'API répond-elle ?

Format d'une trace : `[TRACE][CATÉGORIE] fichier.py:ligne - message | clé=valeur …`. Catégories : REQUEST, CONTEXT, VALIDATION, UNDERSTANDING, ROUTING, REFERENCES, PROCESSING, SEARCH, EMBEDDING, LLM, RESPONSE, ERROR. Pour les couper : `TRACING_ENABLED = False` dans `app/core/tracing.py`.

10.2 Incidents fréquents

Symptôme	Cause probable	Que faire
« Le chatbot est momentanément saturé » (503)	LLM ou Voyage injoignable, quota épuisé, clé invalide, fournisseur surchargé.	Lire la trace `[ERROR]` (cause précise). Vérifier les clés et le quota ; réessayer plus tard si le fournisseur répond 503 ou 429.
« Le service de données est momentanément indisponible » (503)	PostgreSQL arrêté ou en redémarrage.	`docker compose ps`, `docker compose logs db`.
Conteneurs qui redémarrent seuls, requêtes coupées	WSL s'arrête quand plus aucune session ne l'utilise.	Garder une session WSL ouverte, ou régler `vmIdleTimeout` dans `.wslconfig`.
L'API ne démarre pas, init-db en erreur	Migration SQL en échec, ou Voyage injoignable pendant le seed.	`docker logs atm_init` ; corriger puis `docker compose up -d`.
« could not select device driver » au démarrage	`gpus: all` sans GPU NVIDIA.	Retirer `gpus: all` du service `web`.
Aucun cas trouvé alors qu'il en existe	Mauvais type d'équipement, modèle cité absent de `device_model`, seuil trop haut, embeddings manquants.	Vérifier GAB/TPE et le nom de modèle ; consulter les scores dans la trace SEARCH ; ajuster le seuil.
Page admin : 403	`ADMIN_API_KEY` vide ou différente.	Définir la clé dans `.env` et la section `environment`, redémarrer `web`.
« Service de données indisponible » (503) alors que la base tourne	`technician_id` inexistant : la clé étrangère est refusée par PostgreSQL.	Utiliser un technicien existant (table `technician`).
Accents corrompus	Fichier non enregistré en UTF-8.	Toujours enregistrer en UTF-8.

10.3 Sauvegarder et restaurer la base

```
# Sauvegarde
docker exec atm_db pg_dump -U postgres -d atm_chatbot -Fc > atm_chatbot_AAAA-MM-JJ.dump

# Restauration (base existante)
docker exec -i atm_db pg_restore -U postgres -d atm_chatbot --clean --if-exists < atm_chatbot_AAAA-MM-JJ.dump
```

Les données vivent dans le volume Docker `pgdata`. **Ne jamais lancer `docker compose down -v`** sans sauvegarde : l'option `-v` supprime ce volume.

10.4 Tâches courantes

- Recalculer les statistiques : `docker exec atm_app python -m app.jobs.recompute_stats` (prévu pour un CRON).
- Traiter les propositions et ajouts en attente sur `admin.html`.
- Suivre la consommation et le budget sur les consoles Anthropic (ou Google) et Voyage AI.
- Changer une clé : modifier `.env`, puis `docker compose up -d web`.

11. Qualité et tests

Les tests simulent les appels au LLM, à Voyage et à la base : ils ne coûtent rien et ne demandent pas de réseau.

```
# Tous les tests, dans l'image du backend (toutes les dépendances y sont)
docker run --rm -v "$PWD":/src -w /src --entrypoint python atm-chatbot-web -B -m unittest discover -
s tests

# Un fichier
python -B -m unittest discover -s tests -p test_multi_symptom_search.py -v

# Test frontend
node --experimental-default-type=module --test tests/test_external_services_frontend.mjs
```

Fichier	Ce qui est vérifié
`test_conversation_llm.py`	Compréhension, conversation, mémoire des résultats.
`test_multi_symptom_search.py`	Symptômes numérotés, un embedding par symptôme, regroupement et classement des cas.
`test_case_summary_prompt.py`	Consignes anti-invention des prompts de synthèse.
`test_external_services.py`, `test_llm_providers.py`	Erreurs des fournisseurs → 503, pas de repli caché.
`test_solution_proposal_service.py`	Propositions : contexte, création, approbation, rejet.
`test_intervention_add_service.py`	Ajouts d'intervention : règles, aperçu, approbation.
`test_text_validation_service.py`	Filtre local du bruit.
`test_tracing.py`	Traces présentes et sans donnée sensible.
`test_admin_catalog.py`, `test_quick_replies.py`, `test_error_code_search.py`	Administration et menus rapides.
`test_frontend_static_files.py`	Fichiers du frontend servis.

État au 22/10/2026 : 151 tests sur 152 passent dans l'image du backend ; `test_admin_catalog.py` ne se charge pas car `pytest` n'est pas installé dans l'image. Les fichiers `test_embeddings.py` et `test_retrieval.py` ne contiennent pas de test.

Règle : toute évolution du backend s'accompagne d'un test, et la suite complète doit passer avant de fusionner.

12. Guide du développeur

12.1 Conventions

- Noms explicites, compréhensibles par un nouvel arrivant ; commentaires en français, qui expliquent le **pourquoi** et pas le quoi.
- Fichiers en UTF-8.
- Une route n'a pas de logique ; un service ne dépend pas de FastAPI ; erreurs métier dans `app/core/errors.py`.
- Jamais de donnée sensible dans `trace()` : tailles, compteurs et durées seulement.
- Branches Git : développement sur une branche, fusion vers `main` après tests.

12.2 Recettes

Je veux…	Étapes
Ajouter une route	Schéma Pydantic dans `app/schemas/` → logique dans un service → route dans `app/api/routes/` → `include_router` dans `main.py` → si la route est sous `/api/…` d'un nouveau préfixe, l'ajouter à `BACKEND_PATHS_WITH_API_PREFIX` du proxy → test.
Modifier le schéma de la base	Nouvelle migration `NNNN_*.sql` (ne jamais modifier une migration déjà appliquée) → modèle SQLAlchemy dans `app/models/` → seed si besoin.
Modifier un prompt	Prompt de compréhension : garder la consigne système fixe (cache) et le format JSON en accord avec `UnderstoodMessage`. Prompt de synthèse : `_summarize_cases`. Mettre à jour les tests qui vérifient les phrases imposées (« Aucune intervention correspondant… »), reconnues par le backend.
Changer de LLM	`LLM_PROVIDER` dans `.env`. Nouveau fournisseur : une classe `LLMProvider` et une branche dans `get_llm_provider()`.
Changer de modèle d'embedding	Classe `EmbeddingProvider` + `_resolve_provider()`. **Tous les vecteurs doivent être recalculés** (les vecteurs de deux modèles ne se comparent pas) et la dimension de la colonne adaptée par migration.
Régler la pertinence	`SYMPTOM_SIMILARITY_THRESHOLD` (plus haut = moins de cas, plus sûrs) ; nombre de cas et de candidats dans `find_similar_cases`.
Ajouter un menu rapide	Sans code : depuis `admin.html` (codes d'erreur, pannes fréquentes).

12.3 Pièges connus

1. `DATABASE_URL` doit commencer par `postgresql+psycopg2://` ; le seed retire `+psycopg2` pour son propre accès.
2. `app/main.py` doit importer le package `app.models` entier, sinon les relations SQLAlchemy ne trouvent pas leurs classes.
3. Après une erreur SQL, il faut annuler (savepoint ou rollback), sinon toute requête suivante échoue (`InFailedSqlTransaction`).
4. La colonne `embedding` n'est pas mappée dans l'ORM : lecture et écriture en SQL direct (`store_symptom_embedding`).
5. Le cache de prompt ne fonctionne que si le début de la requête est identique au caractère près.

13. Sécurité et conformité

13.1 Ce qui est en place

- Clés API uniquement côté serveur, dans `.env` (exclu de Git par `.gitignore`).
- Routes d'administration protégées par `X-Admin-Key`, comparée à temps constant ; administration désactivée si la clé est vide.
- Réponses d'erreur neutres : ni clé, ni prompt, ni requête SQL exposés.
- Traces sans contenu des messages.
- Le frontend n'exécute jamais le HTML renvoyé par le modèle.
- Une conversation ne peut recevoir de proposition que de son technicien.

13.2 Limites à traiter avant une mise en production

#	Constat	Recommandation
1	Pas d'authentification : le `technician_id` est envoyé par le navigateur et n'est pas vérifié.	Authentification (SSO de l'entreprise ou identifiants) ; le serveur déduit le technicien de la session.
2	Une seule clé admin partagée ; le nom du réviseur est déclaratif.	Comptes administrateurs nominatifs et journal d'audit.
3	`POST /catalog/actions/resolve` modifie le catalogue sans clé admin.	Le protéger par `require_admin` ou le retirer s'il n'est plus utilisé.
4	HTTP sans chiffrement ; mot de passe PostgreSQL par défaut dans `docker-compose.yml` ; port 5432 publié.	HTTPS (reverse proxy), secrets hors du fichier Compose, base non exposée.
5	Les descriptions de pannes sont envoyées à des services d'IA externes (Anthropic ou Google, Voyage AI).	Valider avec le DPO (RGPD, clauses des fournisseurs, durée de conservation) ; ne pas saisir de données personnelles de clients.
6	Le paquet de transfert contient les clés, en clair sur le réseau.	Réserver au réseau de confiance ; à terme, une clé par poste ou un serveur partagé.
7	Versions des dépendances Python non figées (`requirements.txt`).	Figer les versions (`pip freeze`) pour des installations reproductibles.

14. Limites connues et feuille de route

14.1 Limites fonctionnelles

- Le code d'erreur cité est mémorisé mais ne filtre ni ne favorise la recherche.
- Le modèle cité filtre par son nom exact : un nom écrit autrement que dans `device_model` élimine tous les cas (le Copilote le signale).
- Les statistiques sont par type d'équipement, pas par modèle.
- Les échanges des menus rapides ne sont pas enregistrés dans la conversation : le LLM ne les voit pas.
- Au-delà de 100 messages, les plus anciens ne sont plus envoyés et le cache Claude cesse de servir pour cette conversation.
- Pas de route pour relire une conversation passée.

14.2 Dette technique

- L'image de base installe PyTorch (CUDA) et `sentence-transformers`, inutiles depuis le passage à Voyage AI : image lourde, et `gpus: all` obligatoire. À retirer.
- Variables de configuration absentes de `docker-compose.yml` (ex. `MAX_SYMPTOMS_PER_MESSAGE`) : seule leur valeur par défaut s'applique.
- Deux fichiers de test vides ; `pytest` manquant dans l'image.
- Schémas de `docs/diagrammes/` en partie antérieurs aux dernières évolutions (recherche par symptôme, Voyage).

14.3 Évolutions proposées

Priorité	Évolution
1	Authentification réelle et comptes administrateurs nominatifs.
2	Déploiement serveur unique (HTTPS, sauvegardes automatiques) au lieu d'une copie par PC.
3	Code d'erreur : bonus de classement pour les cas ayant le même code.
4	Filtre modèle par identifiant résolu, sans éliminer les cas si le modèle est inconnu.
5	Historique des conversations côté serveur.
6	Indicateurs de pilote : temps de diagnostic, taux de résolution au premier passage, part des réponses utiles, coût par conversation (à partir des traces).
7	Allègement de l'image Docker et versions figées.

Annexes

A. Commandes utiles

```
docker compose up -d --build          # démarrer / reconstruire le backend
docker compose ps                     # état
docker compose logs -f web            # traces du backend
docker compose restart web            # relancer l'API
docker exec -it atm_db psql -U postgres -d atm_chatbot   # console SQL
docker exec atm_app python -m app.jobs.recompute_stats   # recalcul des stats
cd frontend; docker compose up -d --build                 # frontend
```

B. Requêtes SQL de contrôle

```
-- Migrations appliquées
SELECT filename, applied_at FROM schema_migrations ORDER BY filename;

-- Symptômes sans vecteur (introuvables par la recherche)
SELECT intervention_symptom_id, raw_text FROM intervention_symptom WHERE embedding IS NULL;

-- Propositions et ajouts en attente
SELECT proposal_id, created_at FROM solution_proposal WHERE status = 'pending';
SELECT intervention_add_id, created_at FROM intervention_add WHERE status = 'pending';

-- Actions les plus fiables pour les GAB
SELECT s.canonical_text, a.canonical_text, st.successes, st.attempts, st.accuracy_score
FROM symptom_action_outcome_stats st
JOIN symptom s ON s.symptom_id = st.symptom_id
JOIN action a ON a.action_id = st.action_id
WHERE st.device_type = 'gab' ORDER BY st.accuracy_score DESC, st.attempts DESC;
```

C. Documents de référence dans le dépôt

Fichier	Contenu
`COMPREHENSION.md`	Le backend expliqué en 10 minutes.
`README.md`	Flux détaillés : chat, erreurs, propositions, ajouts, validation.
`docs/flux-donnees-db-exemple.md`	Un problème suivi table par table, de l'enregistrement à la réponse.
`frontend/README.md`	Pages, structure, voix, mode démo.
`share/README.md`	Transfert du projet vers un autre PC.
`docs/ATM-BOT_presentation.pptx`	Présentation du projet.